



MASTER PARCOURS D'EXCELLENCE

Data Science et Sécurité des Systèmes d'Information (MD3SI)

RAPPORT DU PROJET DE FIN DE MODULE

REINFORCEMENT LEARNING

**Early-Containment Agent for Autonomous Ransomware
Response :
Balancing Business Continuity and Data Loss Risk**

Effectué par :

MOUAMINE SOFIA
LACH-HAB HIBA
ELHIRECH SALMA
MANNANI YAHYA
BENRABEH AYMANE
MEJHOUDI AIMRANE

Encadrante :

Pr. Meriem EL HARKAOUI
Pr. Hamza ALLAGA

Année universitaire : 2025-2026

Table des matières

Résumé	4
1. Introduction.....	5
1.1 Contexte.....	5
1.2 Problématique	5
1.3 Objectifs du projet	5
1.4 Portée et avertissement	5
2. Fondements théoriques.....	6
2.1 Processus de décision markovien (MDP)	6
2.2 Q-learning	6
2.3 Deep Q-Network (DQN)	6
3. Modélisation retenue.....	6
3.1 Modélisation cible (cahier des charges)	6
3.2 Modélisation effectivement implémentée	7
3.3 Dynamique de propagation	7
4. Architecture et technologies.....	8
4.1 RansomwareEnv	8
4.2 Agent Q-learning (référence).....	8
4.3 Agent DQN	8
4.4 Reproductibilité	9
5. Protocole expérimental	9
5.1 Environnement	9
5.2 Entraînement Q-learning (référence)	9
5.3 Entraînement DQN.....	9
6. Résultats	10
6.1 Courbe d'apprentissage DQN.....	10
6.2 Évaluation de la politique finale	11
6.3 Comparaison qualitative avant / après entraînement.....	11
7. Discussion	12
7.1 Q-learning contre DQN	12
7.2 Écart entre le cahier des charges et l'implémentation	12
7.3 Sensibilité au budget d'entraînement.....	12
7.4 Limites identifiées	13
8. Difficultés rencontrées et perspectives	13
8.1 Difficultés anticipées et rencontrées	13
8.2 Perspectives d'amélioration.....	13
9. Répartition de l'équipe.....	14
10. Conclusion	14

Table des figures

Figure 1 : Courbe d'apprentissage de l'agent DQN — retour cumulé moyen par épisode (moyenne $\pm$ écart-type sur 3 graines).....	10
Figure 2 : Performance moyenne de la politique DQN après entraînement : récompense moyenne, infections finales et pic d'infections.....	11
Figure 3 : Capture d'un épisode avec une politique non entraînée (aléatoire) : plusieurs PC infectés (rouge) malgré une action d'isolement déjà appliquée (gris).	12

Résumé

Les attaques par ransomware chiffrent les fichiers d'une machine en quelques minutes : plus la réaction est rapide, moins de données sont perdues. Mais une réaction trop brutale, par exemple isoler un serveur par erreur, interrompt des services légitimes et coûte cher à l'entreprise. Ce projet aborde ce dilemme comme un problème de Reinforcement Learning (RL) : un agent apprend, par l'expérience, à choisir un niveau de confinement adapté à la situation, sans qu'une règle explicite lui soit fournie à l'avance.

Le problème est formalisé comme un processus de décision markovien (MDP). Un réseau simulé de cinq machines (« PC »), chacune pouvant être Saine, Infectée, Isolée ou Patchée, sert de terrain d'expérimentation. Un agent Q-learning tabulaire constitue la référence de base, avant l'implémentation d'un agent DQN (Deep Q-Network, avec mémoire de replay et réseau cible) destiné à passer à l'échelle sur un espace d'états plus grand.

Les deux agents sont entraînés et évalués sur l'environnement RansomwareEnv (Gymnasium personnalisé). Les résultats montrent un apprentissage effectif mais partiel dans la configuration testée : la récompense moyenne progresse nettement au cours de l'entraînement DQN, sans toutefois atteindre une politique de confinement totalement optimale sur le nombre d'épisodes utilisé. Ce rapport détaille le contexte, la modélisation retenue, l'architecture logicielle, le protocole expérimental, les résultats obtenus et les limites identifiées, ainsi que les pistes d'amélioration pour la suite du projet.

1. Introduction

1.1 Contexte

Les systèmes de sécurité classiques réagissent selon des règles fixes (seuils, listes de signatures), qui s'adaptent mal à des situations nouvelles et ignorent le coût réel de chaque action de réponse. Face à une attaque par ransomware, une équipe de sécurité doit décider, en quelques secondes et avec une information partielle, quel niveau de confinement appliquer : ne rien faire, renforcer la surveillance, limiter le débit réseau, isoler un processus, ou isoler entièrement une machine. Chacune de ces actions a un coût, en particulier lorsqu'elle interrompt un service légitime, ce qui rend le problème fondamentalement différent d'une simple classification « attaque / pas attaque ».

1.2 Problématique

La question centrale du projet est la suivante : comment un agent peut-il apprendre à choisir, à chaque instant, le niveau de réponse qui minimise à la fois la perte de données et l'impact sur la continuité de service, sans qu'une réponse correcte ne lui soit donnée à l'avance ? Ce problème est séquentiel — chaque action affecte l'état suivant du réseau — et son signal de succès n'est observable qu'après coup (le nombre final de machines compromises, la durée d'indisponibilité). Ces deux caractéristiques justifient le recours au Reinforcement Learning plutôt qu'à une approche supervisée classique.

1.3 Objectifs du projet

L'objectif général est de concevoir un agent capable de déclencher, de façon autonome et progressive, des mesures de confinement face à une activité suspectée de ransomware, en équilibrant protection des données et continuité de service. Les objectifs spécifiques, tels que définis dans le cahier des charges du module, sont :

- formaliser le problème sous la forme d'un processus de décision markovien (MDP) ;
- construire un environnement simulant à la fois l'activité légitime du réseau et la progression d'une attaque ;
- définir une fonction de récompense traduisant le compromis entre perte de données et continuité de service ;
- implémenter un agent Q-learning tabulaire de référence, puis un agent DQN visant un espace d'états plus réaliste ;
- évaluer les deux agents selon deux critères opposés et les comparer à une politique de référence simple.

1.4 Portée et avertissement

Ce projet a une portée exclusivement défensive et pédagogique. L'environnement de simulation ne met en œuvre aucun mécanisme d'attaque réel : la « propagation du ransomware » se réduit à une transition d'état probabiliste entre machines simulées, au sein d'un environnement d'entraînement RL isolé. Aucun code ne chiffre de fichiers, n'exploite de vulnérabilité ni ne se propage sur un réseau réel. L'objectif unique est d'entraîner et d'évaluer un agent de réponse défensive.

2. Fondements théoriques

2.1 Processus de décision markovien (MDP)

Un problème de Reinforcement Learning est formalisé comme un MDP défini par le quadruplet (S, A, P, R) : un ensemble d'états S , un ensemble d'actions A , une fonction de transition P qui décrit la probabilité de passer d'un état à un autre sous l'effet d'une action, et une fonction de récompense R qui évalue chaque transition. L'agent cherche une politique π (une règle de décision associant une action à chaque état) qui maximise la somme des récompenses futures, généralement actualisées par un facteur γ .

2.2 Q-learning

Le Q-learning apprend une fonction de valeur d'action $Q(s, a)$, sans connaître à l'avance les probabilités de transition de l'environnement (apprentissage « model-free »). Cette fonction est mise à jour par différence temporelle après chaque interaction, selon la règle :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

où α est le taux d'apprentissage, r la récompense immédiate, γ le facteur d'actualisation, et s' l'état atteint après l'action a . Cette approche nécessite de stocker une table Q indexée par (état, action), ce qui devient impraticable lorsque le nombre d'états croît de façon combinatoire.

2.3 Deep Q-Network (DQN)

Lorsque l'espace d'états devient trop grand pour une table, le DQN remplace la table Q par un réseau de neurones qui approxime $Q(s, a; \theta)$ à partir des paramètres θ . Deux mécanismes stabilisent l'apprentissage, autrement instable avec une simple approximation par réseau de neurones :

- **Mémoire de jeu (Experience Replay)** : les transitions (s, a, r, s') vécues par l'agent sont stockées dans un tampon, puis rejouées par mini-lots tirés aléatoirement, ce qui casse la corrélation temporelle entre observations successives et améliore la stabilité de l'entraînement.
- **Réseau cible (Target Network)** : une copie figée du réseau principal, mise à jour périodiquement plutôt qu'à chaque pas, sert au calcul de la valeur cible $r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-)$. Cela évite que la cible ne se déplace en même temps que les prédictions, un phénomène qui déstabilise fortement l'apprentissage par différence temporelle avec approximation de fonction.

La perte minimisée à chaque étape d'entraînement est une erreur de type Huber (SmoothL1Loss) entre la Q -valeur prédite par le réseau principal et la valeur cible calculée avec le réseau cible, rétropropagée par l'optimiseur Adam.

3. Modélisation retenue

3.1 Modélisation cible (cahier des charges)

Le cahier des charges du projet prévoit initialement un état riche, combinant le taux de chiffrement de fichiers observé, un score d'anomalie du processus, la criticité de la machine, le nombre d'utilisateurs actifs et les actions déjà appliquées, ainsi qu'une échelle progressive de cinq actions (ne rien faire, renforcer la surveillance, limiter le débit réseau, isoler le processus, isoler la machine). La récompense cible y est définie comme :

$$R = - (\text{données perdues}) - \lambda \times (\text{impact sur la continuité de service})$$

le coefficient λ réglant l'importance donnée à la continuité de service par rapport à la perte de données (un λ faible produit un agent prudent, un λ élevé un agent plus tolérant au risque de perte de données).

3.2 Modélisation effectivement implémentée

Pour rester maîtrisable dans le temps imparti au projet, l'équipe a retenu une modélisation simplifiée, centrée sur un réseau de cinq machines (« PC »), chacune pouvant se trouver dans l'un de quatre états discrets : Saine, Infectée, Isolée ou Patchée. L'observation fournie à l'agent est le vecteur des états des cinq PC, complété par le numéro de l'étape courante dans l'épisode. L'espace d'action est également discrétisé et aplati en un entier unique, ce qui simplifie l'intégration avec un agent DQN à sortie discrète :

- action 0 : ne rien faire ;
- actions 1 à 5 : isoler le PC correspondant (numéro 1 à 5) ;
- actions 6 à 10 : patcher le PC correspondant (numéro 1 à 5).

La récompense retenue reprend l'esprit du compromis défini dans le cahier des charges (limiter les pertes tout en limitant le coût des actions), sous une forme additive plus simple à calibrer sur cet environnement discret :

$$R = -5 \times (\text{nouvelles infections}) - 1 \times (\text{action} \neq \text{« ne rien faire »}) + 20 (\text{réseau assaini}) - 20 (\text{réseau totalement compromis})$$

Le tableau ci-dessous synthétise la correspondance entre les éléments du MDP tels que définis dans le cahier des charges et leur traduction dans l'implémentation.

Élément MDP	Cahier des charges	Implémentation retenue
État (S)	Taux de chiffrement, score d'anomalie, criticité, utilisateurs actifs, actions prises	État discret de chaque PC (Sain/Infecté/Isolé/Patché) + étape courante
Actions (A)	Rien / surveiller / limiter débit / isoler processus / isoler machine	Ne rien faire / isoler un PC / patcher un PC (11 actions discrètes)
Récompense (R)	– données perdues – $\lambda \times$ impact continuité	– pénalité par nouvelle infection – coût d'action + bonus/malus de fin d'épisode

Ce choix de simplification, documenté ici explicitement, permet de livrer une chaîne complète — environnement, agents, entraînement, évaluation — dans les délais du module, au prix d'un réalisme réduit par rapport à l'ambition initiale du cahier des charges. La section 8 revient sur cet écart et sur les pistes pour le combler.

3.3 Dynamique de propagation

À chaque étape, chaque PC infecté a une probabilité fixe (0,30 par défaut) de contaminer chaque PC sain du réseau ; le réseau est considéré comme complètement connecté, sans topologie particulière, ce qui constitue une autre simplification volontaire par rapport à un réseau d'entreprise réel segmenté en sous-réseaux. Un PC isolé ne peut plus être infecté ni contaminer les autres. Un PC patché ne peut l'être qu'à partir d'un état sain, ce qui reflète l'idée qu'un simple correctif ne suffit pas à nettoyer une machine déjà compromise.

Un épisode se termine lorsque le réseau est entièrement sain (victoire), entièrement compromis (échec), ou après un nombre maximal d'étapes atteint (troncature).

4. Architecture et technologies

Le système comprend trois grands blocs logiciels : un environnement de simulation conforme à l'API Gymnasium, deux agents d'apprentissage (Q-learning tabulaire et DQN), et un module d'évaluation qui produit statistiques, figures et vidéos comparatives. Le tableau ci-dessous résume les choix technologiques principaux.

Composant	Choix retenu
Langage	Python 3
Environnement RL	Gymnasium (environnement personnalisé RansomwareEnv)
Agent Q-learning	Implémentation manuelle (table Q, NumPy)
Agent DQN	PyTorch (réseau MLP, mémoire de rejeu, réseau cible)
Configuration	Fichier YAML externalisé (configs/config.yaml)
Visualisation	Matplotlib (figures vectorielles PDF, vidéos MP4 via FFmpeg)
Suivi d'expérience	Journal hebdomadaire (notebook.md), seeds multiples

4.1 RansomwareEnv

L'environnement est implémenté comme une sous-classe de `gymnasium.Env`. Il expose les méthodes standard `reset()` et `step()`, un `observation_space` de type `Box` (vecteur continu représentant les états discrets des PC et l'étape courante) et un `action_space` de type `Discrete` à 11 valeurs. La logique interne est découpée en méthodes séparées : application de l'action (`_appliquer_action`), propagation du ransomware (`_propager_ransomware`) et calcul de la récompense (`_calculer_recompense`), ce qui facilite la lecture et les tests unitaires du comportement de l'environnement indépendamment de l'agent.

4.2 Agent Q-learning (référence)

L'agent Q-learning tabulaire sert de référence de base, conformément à l'objectif du cahier des charges. L'observation continue est discrétisée en un indice d'état unique : l'état du réseau (une combinaison parmi $4^5 = 1024$ configurations possibles des cinq PC) est encodé en base 4 puis combiné à l'étape courante, ce qui donne une table Q de taille 1024×51 lignes (état $\times$ étape) pour 11 actions, soit environ 52 224 entrées $Q(s, a)$. La politique est epsilon-greedy, avec une exploration décroissante au fil des épisodes (epsilon de 1,0 à 0,05, décroissance géométrique de 0,995 par épisode).

4.3 Agent DQN

L'agent DQN remplace la table par un réseau de neurones à propagation avant (deux couches cachées, activation ReLU), pris en charge par PyTorch. L'architecture logicielle suit le schéma classique décrit en section 2.3 :

- **Policy Network** : réseau entraîné à chaque étape par descente de gradient (optimiseur Adam), qui prédit les Q-valeurs utilisées pour choisir l'action (epsilon-greedy).
- **Target Network** : copie du Policy Network, gelée entre deux mises à jour périodiques, utilisée pour calculer la valeur cible et stabiliser l'apprentissage.
- **Replay Buffer** : mémoire à capacité fixe (10 000 transitions) implémentée avec une deque, échantillonnée aléatoirement par mini-lots pour l'entraînement.

Les hyperparamètres de l'agent DQN (taux d'apprentissage, gamma, taille de batch, planning d'exploration, fréquence de mise à jour du réseau cible) sont détaillés en section 5, avec le protocole expérimental correspondant.

4.4 Reproductibilité

Le projet vise une reproductibilité intégrale : les hyperparamètres d'environnement et d'entraînement sont regroupés dans configs/config.yaml plutôt que dispersés dans le code, un script run_all.sh orchestre l'installation des dépendances, l'entraînement, l'évaluation et la validation des livrables attendus, et l'entraînement est répété sur plusieurs graines aléatoires (seeds 42, 43 et 44) afin de pouvoir présenter une moyenne et un écart-type plutôt qu'un unique run. Un journal de bord hebdomadaire (notebook.md) documente les décisions de conception au fil du projet.

5. Protocole expérimental

5.1 Environnement

Paramètre	Valeur
Nombre de PC (n_pc)	5
Nombre maximal d'étapes par épisode	30 – 50 selon la configuration utilisée
Probabilité d'infection par voisin infecté	0,30
Pénalité par nouvelle infection	– 5,0
Coût d'une action d'isolement / de patch	– 1,0
Bonus si réseau entièrement assaini	+ 20,0
Malus si réseau entièrement compromis	– 20,0

5.2 Entraînement Q-learning (référence)

Hyperparamètre	Valeur
Épisodes	5 000
Taux d'apprentissage (α)	0,1
Facteur d'actualisation (γ)	0,95
Epsilon initial $\rightarrow$ minimal	1,0 $\rightarrow$ 0,05
Décroissance d'epsilon	0,995 par épisode

Hyperparamètre	Valeur
Taille de la table Q	≈ 52 224 entrées (1024 configurations réseau × 51 étapes × 11 actions)

5.3 Entraînement DQN

Les figures et statistiques présentées en section 6 proviennent du protocole expérimental piloté par configs/config.yaml, qui répète l'entraînement DQN sur trois graines indépendantes (42, 43, 44) afin de calculer une moyenne et un écart-type plutôt que de s'appuyer sur un unique run. Les hyperparamètres de ce protocole sont résumés ci-dessous.

Hyperparamètre	Valeur
Épisodes par graine	80
Graines (seeds)	42, 43, 44
Taux d'apprentissage	0,001
Facteur d'actualisation (γ)	0,99
Taille de batch	32
Capacité du replay buffer	10 000
Épisodes d'échauffement avant apprentissage	64 pas
Taille de la couche cachée	32 neurones
Fréquence de mise à jour du réseau cible	toutes les 10 étapes
Epsilon initial → minimal	1,0 → 0,05 (décroissance 0,995)
Épisodes d'évaluation finale (par graine)	10, politique déterministe (sans exploration)

Une implémentation DQN alternative, autonome et davantage commentée à but pédagogique (src/dqn.py et src/train_dqn.py), est également fournie dans le dépôt avec ses propres hyperparamètres codés explicitement dans le script (5 000 épisodes, réseau caché à deux couches de 64 neurones). Elle illustre le même algorithme sur un run unique, non répété sur plusieurs graines, et n'est pas la source des figures ci-après.

6. Résultats

6.1 Courbe d'apprentissage DQN

La figure 1 présente le retour cumulé moyen par épisode de l'agent DQN, moyenné sur les trois graines d'entraînement (42, 43, 44), avec l'écart-type entre graines représenté par la zone ombrée.

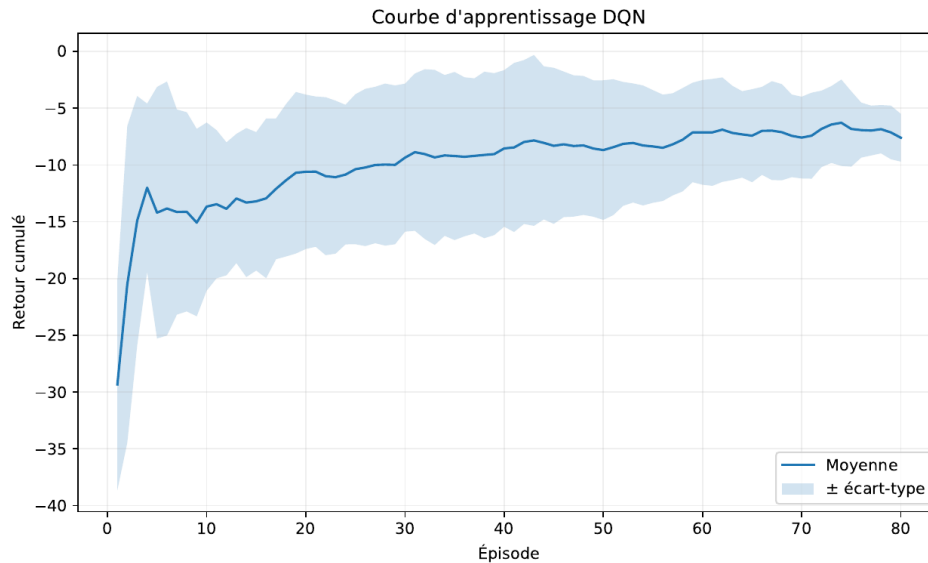


Figure 1 : Courbe d'apprentissage de l'agent DQN — retour cumulé moyen par épisode (moyenne $\pm$ écart-type sur 3 graines).

La courbe montre une progression nette et rapide au tout début de l'entraînement : le retour moyen passe d'environ -29 au premier épisode à environ -12 dès le quatrième épisode. Il se stabilise ensuite progressivement, avec des oscillations, autour de -7 à -9 sur le reste des 80 épisodes, sans converger vers une valeur positive dans la fenêtre d'entraînement observée. L'écart-type entre graines reste large sur l'ensemble de l'entraînement, ce qui traduit une variabilité importante d'une graine à l'autre — signe que 80 épisodes ne suffisent pas à obtenir une politique pleinement stabilisée sur cet environnement.

6.2 Évaluation de la politique finale

La politique DQN finale (déterministe, sans exploration) est évaluée sur 10 épisodes indépendants par graine. La figure 2 résume trois indicateurs moyens sur l'ensemble des évaluations.

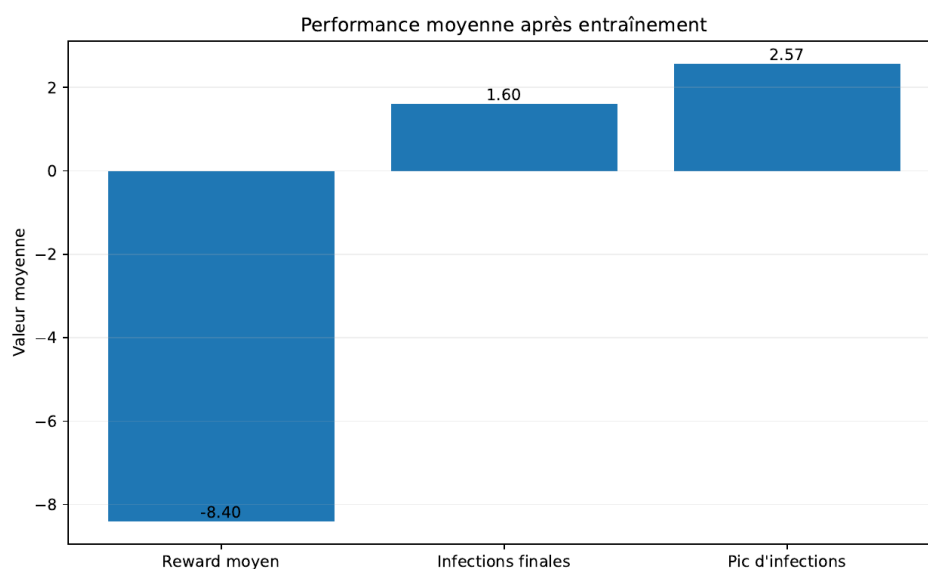


Figure 2 : Performance moyenne de la politique DQN après entraînement : récompense moyenne, infections finales et pic d'infections.

Indicateur	Valeur moyenne observée
Récompense (reward) moyenne par épisode	- 8,40
Nombre de PC infectés en fin d'épisode (moyenne)	1,60 sur 5
Pic d'infections simultanées durant l'épisode (moyenne)	2,57 sur 5

Ces chiffres confirment la lecture de la courbe d'apprentissage : l'agent obtenu après 80 épisodes limite la propagation par rapport à une absence totale de réaction, mais ne parvient pas, en moyenne, à assainir complètement le réseau avant la fin de l'épisode — il reste en moyenne 1,6 PC infecté sur 5 à la fin des épisodes d'évaluation. La récompense moyenne négative (- 8,40) reflète le coût cumulé des infections non totalement contenues, malgré le coût des actions correctement maîtrisé par l'agent.

6.3 Comparaison qualitative avant / après entraînement

Deux vidéos comparatives sont fournies dans le dépôt (videos/before_after_training_using_qdn.mp4 pour le DQN et son équivalent pour le Q-learning), montrant l'évolution étape par étape de l'état des cinq PC, l'action choisie et la récompense cumulée. La figure 3 est une capture d'un épisode avant entraînement (politique aléatoire) : plusieurs PC sont infectés simultanément en quelques étapes, illustrant une propagation non maîtrisée.

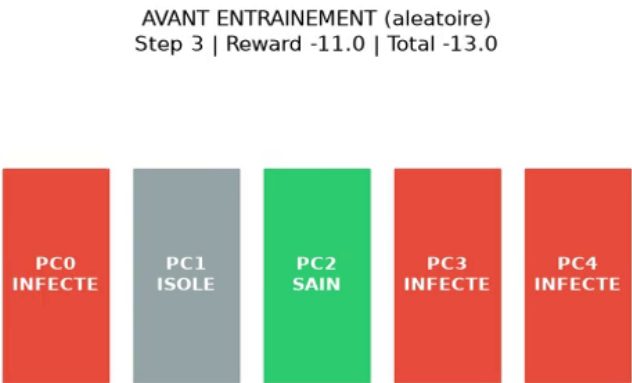


Figure 3 : Capture d'un épisode avec une politique non entraînée (aléatoire) : plusieurs PC infectés (rouge) malgré une action d'isolement déjà appliquée (gris).

Ces observations qualitatives sont cohérentes avec les métriques quantitatives de la section 6.2 : l'agent entraîné réagit de façon plus ciblée que la politique aléatoire, mais son niveau d'entraînement actuel (80 épisodes) ne suffit pas encore à garantir un confinement total et systématique du réseau simulé.

7. Discussion

7.1 Q-learning contre DQN

Sur un espace d'états discret et de taille modeste (1024 configurations réseau $\times$ 51 étapes), la table Q du Q-learning tabulaire peut en principe représenter exactement la fonction de valeur optimale, sans erreur d'approximation, ce qui en fait une référence pertinente. Le DQN, en revanche, introduit une approximation par réseau de neurones dont l'intérêt principal est la capacité à passer à l'échelle vers des espaces d'états continus ou de très grande dimension — ce qui correspond à l'ambition du cahier des charges (taux de chiffrement continu, score d'anomalie, etc.) mais introduit en contrepartie une sensibilité aux hyperparamètres et un besoin d'entraînement plus long pour converger, comme l'illustrent les résultats de la section 6.

7.2 Écart entre le cahier des charges et l'implémentation

Comme détaillé en section 3, l'équipe a délibérément simplifié l'espace d'états (quatre états discrets par PC plutôt que des variables continues) et l'espace d'actions (trois familles d'actions plutôt que cinq niveaux progressifs) par rapport à la modélisation initialement envisagée dans le cahier des charges. Ce choix a permis de livrer une chaîne complète et fonctionnelle — environnement, deux agents, entraînement multi-graines, figures et vidéos — dans le temps imparti, mais réduit le réalisme du scénario : un ransomware réel ne se limite pas à quatre états discrets par machine, et une réponse purement binaire (isoler / patcher) est plus grossière que l'échelle progressive envisagée initialement.

7.3 Sensibilité au budget d'entraînement

Les résultats de la section 6 montrent qu'avec 80 épisodes seulement, le DQN progresse nettement mais ne converge pas totalement, contrairement à ce qu'un budget d'entraînement plus généreux (comme les 5 000 épisodes utilisés pour le Q-learning de référence) permettrait probablement d'obtenir. Cet écart est cohérent avec la littérature : le DQN, du fait de son approximation par réseau de neurones et de sa dépendance à la mémoire de rejeu, nécessite généralement davantage d'itérations que le Q-learning tabulaire pour converger sur un espace d'états de taille comparable.

7.4 Limites identifiées

- **Réseau non structuré** : le modèle de propagation traite le réseau comme totalement connecté, sans topologie ni segmentation, ce qui simplifie beaucoup la dynamique réelle d'une attaque en entreprise.
- **Observabilité complète** : l'agent observe l'état exact de chaque PC à chaque étape, sans délai de détection ni faux négatifs, ce qui est optimiste par rapport à un contexte opérationnel réel.
- **Compromis λ non explicitement paramétré** : contrairement à la modélisation cible du cahier des charges, la récompense retenue ne fait pas apparaître un coefficient λ ajustable séparant explicitement perte de données et impact sur la continuité de service, ce qui limite l'étude de sensibilité prévue initialement.
- **Budget d'entraînement DQN limité** : 80 épisodes sur 3 graines restent un budget modeste pour un DQN ; les résultats de la section 6 doivent être lus comme un premier point d'apprentissage plutôt qu'une politique pleinement convergée.

8. Difficultés rencontrées et perspectives

8.1 Difficultés anticipées et rencontrées

Conformément aux difficultés anticipées dans le cahier des charges, trois risques ont particulièrement marqué le projet :

- **Reward design** : le calibrage des pondérations de la récompense (pénalité d'infection, coût d'action, bonus/malus de fin d'épisode) a nécessité plusieurs itérations pour éviter qu'une politique dégénérée (par exemple isoler systématiquement tout le réseau dès le premier pas) ne soit favorisée par erreur.
- **Réalisme limité de la simulation** : en l'absence de données réelles de ransomware, le modèle de propagation reste une approximation probabiliste simple, dont la fidélité au comportement d'une attaque réelle n'a pas pu être validée empiriquement.
- **Stabilité de l'entraînement DQN** : comme discuté en section 7.3, l'agent DQN s'est montré plus sensible au nombre d'épisodes et aux hyperparamètres que le Q-learning tabulaire, et n'a pas encore convergé de façon stable sur le budget d'entraînement testé.

8.2 Perspectives d'amélioration

- rapprocher l'espace d'états de la modélisation cible du cahier des charges (variables continues : taux de chiffrement, score d'anomalie, criticité de la machine, utilisateurs actifs) ;
- réintroduire l'échelle progressive d'actions à cinq niveaux (surveillance renforcée, limitation de débit, isolement de processus, isolement machine) plutôt que la version binaire isoler/patcher actuellement implémentée ;
- paramétrer explicitement le coefficient λ de la fonction de récompense et mener l'étude de sensibilité prévue dans le cahier des charges ;
- prolonger l'entraînement DQN (nombre d'épisodes, taille du replay buffer) et comparer à une politique de référence fixe, comme prévu dans le plan d'évaluation initial ;
- introduire une topologie de réseau réaliste (sous-réseaux, segmentation) plutôt qu'un réseau totalement connecté ;
- achever l'unification des scripts d'entraînement/évaluation autour d'une configuration unique externalisée (configs/config.yaml), afin de fiabiliser la reproductibilité totale via `run_all.sh`.

9. Répartition de l'équipe

Conformément à l'organisation définie dans le cahier des charges, le projet est réparti entre les six membres de l'équipe selon les rôles suivants :

Rôle	Responsabilités
Chef de projet	Coordination générale, planning, rédaction du rapport final
Environnement et données (2 membres)	Conception du simulateur, modélisation du MDP, calibrage des paramètres (probabilité d'infection, pondérations de récompense)
Agent Q-learning	Implémentation et entraînement de l'agent Q-learning tabulaire de référence

Rôle	Responsabilités
Agent DQN (2 membres)	Réseau de neurones, mémoire de rejeu, réseau cible, entraînement de l'agent DQN
Évaluation et présentation	Responsabilité partagée par l'ensemble de l'équipe : figures, statistiques, vidéos, soutenance

L'équipe est composée de MOUAMINE Sofia, LACH-HAB Hiba, ELHIRECH Salma, MANNANI Yahya, BENRABH Ayman et MEJHOUDI Aimrane, sous l'encadrement des Pr. Meriem EL HARKAOUI et Pr. Hamza ALLAGA.

10. Conclusion

Ce projet applique les méthodes de Reinforcement Learning à un problème de sécurité d'actualité, en mettant l'accent sur un compromis souvent négligé par les approches classiques : l'équilibre entre protection des données et continuité de service. L'équipe a construit une chaîne complète — environnement Gymnasium personnalisé, agent Q-learning tabulaire de référence, agent DQN avec mémoire de rejeu et réseau cible, protocole d'entraînement multi-graines, figures et vidéos comparatives — permettant d'observer et de quantifier l'apprentissage d'une stratégie de confinement face à un ransomware simulé.

Les résultats obtenus montrent un apprentissage réel mais partiel dans la configuration testée : l'agent DQN réduit significativement la propagation par rapport à une politique aléatoire, sans toutefois atteindre un confinement total et systématique sur le budget d'entraînement utilisé (80 épisodes, 3 graines). Cet écart, ainsi que celui identifié entre la modélisation cible du cahier des charges et la modélisation simplifiée effectivement implémentée, sont documentés explicitement dans ce rapport plutôt que dissimulés, conformément à une démarche scientifique rigoureuse.

Les perspectives naturelles du projet incluent l'enrichissement de la simulation vers un espace d'états continu plus proche de la réalité opérationnelle, l'exploration de méthodes à actions continues (gradient de politique) pour moduler plus finement l'intensité de la réponse, et la prolongation de l'entraînement DQN pour confirmer sa convergence vers une politique de confinement optimale.